

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчет по лабораторной работе №2
по дисциплине
«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00	_____ /Макаров С.А./
Руководитель преподаватель	_____ /Шмакова Н.А./

Киров 2026

Цель работы

Цель работы: изучить и освоить принципы объектно-ориентированного программирования, включая инкапсуляцию, наследование и полиморфизм, путем создания иерархии классов в выбранной предметной области, взаимодействие с базой данных и графическим интерфейсом.

Задание

На основании согласованной в первой лабораторной работе предметной области разработать приложение с графическим интерфейсом пользователя, используя классы и особенности C#. Сведения об объектах предметной области необходимо хранить в БД.

Решение

Предметная область описывает управление ассортиментом блюд в заведении общественного питания. Система позволяет хранить информацию о различных блюдах, учитывать их специфические характеристики, рассчитывать итоговую цену для клиента в зависимости от этих характеристик и выводить информацию о блюде в удобном виде.

Программа содержит классы «Dish», представляющий базовый класс блюда, «Pizza», наследованный от «Dish» и представляющий класс пиццы, «Salad» наследованный от «Dish» и представляющий класс салата.

Базовый класс «Dish» содержит:

- Свойства:
 - `public Guid Id { get; }` – свойство, означающее уникальный идентификатор;
 - `public string Name { get; set; }` – свойство, означающее название блюда, строковый тип данных;
 - `public int Weight { get; set; }` – свойство, означающее вес блюда, целочисленный тип данных;

- `public double Price { get; set; }` – свойство, цена блюда, формат с плавающей запятой;
- Конструкторы:
 - `public Dish(string name, int weight = 100, double price = 10.0)` - публичный конструктор, принимает в качестве параметров название, вес и цену блюда;
 - `public Dish(string name) : this(name, 200, 20.0)` - публичный конструктор, принимает название блюда в качестве параметра;
 - `public Dish() : this("Dish")` – публичный конструктор, задает значения полей по умолчанию;
- Абстрактные методы:
 - `public abstract string DisplayInfo()` – публичный абстрактный метод, возвращающий информацию о блюде, не принимает параметров;
 - `public abstract double CalcFullPrice()` – публичный абстрактный метод, возвращающий полную цену блюда.

Класс «Pizza», наследованный от класса «Dish» содержит:

- Собственные свойства:
 - `public Dough Dough { get; set; } = Dough.Thick` – свойство, означающее типа теста, по умолчанию Thick;
- Конструкторы:
 - `public Pizza() : base()` – публичный конструктор, задает значения полей по умолчанию;
 - `public Pizza(string name) : base(name)` – публичный конструктор, принимает название пиццы в качестве параметра;
 - `public Pizza(string name, int weight, double price) : base(name, weight, price)` – публичный конструктор, принимает в качестве параметров название, вес и цену пиццы;

– Переопределенные методы:

- `public override string DisplayInfo()` – публичный метод, возвращающий информацию о пицце (название, тип теста, вес, цена), не принимает параметров, возвращает информацию в следующем виде:

`Pizza: Margarita, thick dough, 450g, $18.00;`

- `public override double CalcFullPrice()` – публичный метод, возвращает полную цену в зависимости от типа теста (Thick: $\text{price} * 1.5$, Thin: $\text{price} * 1.3$), не принимает параметров;

– Собственные методы:

- `public void ChangeDough()` – публичный метод, меняет тип теста, не принимает параметров;
- `public void CutInSlices()` – публичный метод, нарезает пиццу на куски, не принимает параметров, возвращает информацию в следующем виде:

`Cutting the pizza into slices...;`

Класс «Salad», наследованный от класса «Dish» содержит:

– Собственные свойства:

- `public Dressing Dressing { get; set; } = Dressing.OliveOil` – свойство, означающее тип заправки, по умолчанию OliveOil;

– Конструкторы:

- `public Salad() : base()` – публичный конструктор, задает значения полей по умолчанию;
- `public Salad(string name) : base(name)` – публичный конструктор, принимает название салата в качестве параметра;
- `public Salad(string name, int weight, double price) : base(name, weight, price)` – публичный конструктор, принимает в качестве параметров название, вес и цену салата;

- Переопределенные методы:
 - `public string DisplayInfo() override` – публичный метод, отображает информацию о салате (название, тип заправки, вес, цена), не принимает параметров, возвращает информацию в следующем виде:
`Salad: Caesar, OliveOil dressing, 250g, $12.60;`
 - `public double CalcFullPrice() override` – публичный метод, возвращает полную цену в зависимости от типа заправки (OliveOil: price * 1.4, Mayonnaise: price * 1.3), не принимает параметров;
- Собственные методы:
 - `public void ChangeDressing()` – публичный метод, меняет тип заправки, не принимает параметров;
 - `public string TossWithDressing()` – публичный метод, перемешивает салат с заправкой, не принимает параметров, возвращает информацию в следующем виде:
`Tossing the salad with OliveOil...`

Диаграмма классов сущностей представлена на рисунке 1.

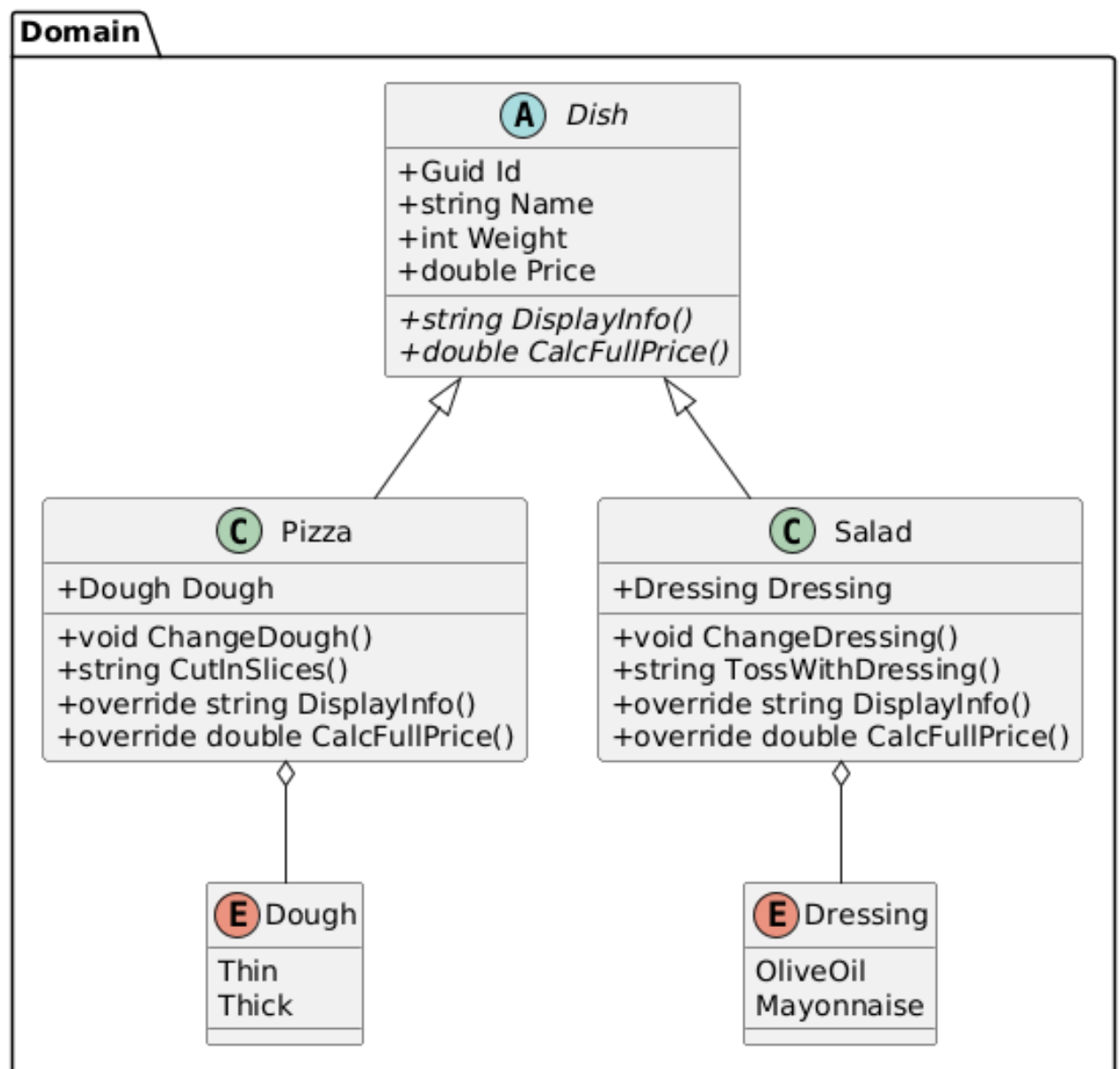


Рисунок 1 – Диаграмма классов сущностей

Разработаны серверные и клиентское приложение в виде веб-интерфейса. Серверное приложение содержит подключение к базе данных, конфигурацию моделей и контроллеры.

Подключение к базе данных Postgres происходит с помощью следующей строки подключения:

```

"ConnectionStrings": {
  "LabaratoryDbContext": "User ID=root;Password=root;
    Host=localhost;Port=5432;Database=labaratory_cs;"
}
  
```

Далее регистрируется контекст общения с базой данных в главном файле:

```
builder.Services.AddDbContext<LabaratoryDbContext>(options =>
{
    options.UseNpgsql(builder.Configuration.GetConnectionString(nameof
}));
```

И сам класс контекста:

```
public class LabaratoryDbContext(
    DbContextOptions<LabaratoryDbContext> options) : DbContext(options)
{
    public DbSet<Pizza> Pizzas { get; set; }
    public DbSet<Salad> Salads { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfiguration(new PizzaConfiguration());
        modelBuilder.ApplyConfiguration(new SaladConfiguration());

        base.OnModelCreating(modelBuilder);
    }
}
```

Конфигурация сущности, например для «Pizza» содержит метод Configure(), в котором описываются поля сущности для базы данных:

```
public class PizzaConfiguration : IEntityConfiguration<Pizza>
{
    public void Configure(EntityTypeBuilder<Pizza> builder)
    {
        builder.HasKey(p => p.Id);

        builder.Property(p => p.Name);
        builder.Property(p => p.Weight);
    }
}
```

```

        builder.Property(p => p.Price);

        builder.Property(p => p.Dough)
            .HasConversion(
                d => d.ToString(),
                d => Enum.Parse<Dough>(d)
            );
    }
}

```

Следующим шагом идёт создание контроллера для ответов на запросы для клиентов. Класс «PizzasController» наследуется от класса «Controller».

Содержит следующие методы:

- `public async Task<IActionResult> CreateAsync([FromBody] CreatePizzaRequest request)` – метод, который создает новую запись о пицце;
- `public async Task<IActionResult> GetAllAsync()` – метод, возвращающий весь список пицц, хранящиеся в базе данных;
- `public async Task<IActionResult> GetByIdAsync([FromRoute] Guid id)` – метод, возвращающий отдельную запись о пицце по уникальному идентификатору;
- `public async Task<IActionResult> CutAsync([FromRoute] Guid id)` – метод, возвращающий сообщение о нарезании пиццы на кусочки по уникальному идентификатору;
- `public async Task<IActionResult> DisplayInfoAsync([FromRoute] Guid id)` – метод, возвращающий полное описание записи пиццы по уникальному идентификатору;
- `public async Task<IActionResult> GetFullPriceAsync([FromRoute] Guid id)` – метод, возвращающий полную стоимость пиццы по уникальному идентификатору;

- `public async Task<IActionResult> UpdateAsync([FromRoute] Guid id, [FromBody] UpdatePizzaRequest request)` – метод, который обновляет записи о пицце по уникальному идентификатору;
- `public async Task<IActionResult> ChangeDressingAsync([FromRoute] Guid id)` – метод, меняющий тип теста пиццы по уникальному идентификатору;
- `public async Task<IActionResult> DeleteAsync([FromRoute] Guid id)` – метод, удаляющий запись о пицце по уникальному идентификатору.

Конфигурация и контроллер для сущности «Salad» выполняется аналогичным способом и содержат аналогичные методы.

Диаграмма классов контроллеров и конфигураций сущностей представлены на рисунке 2.

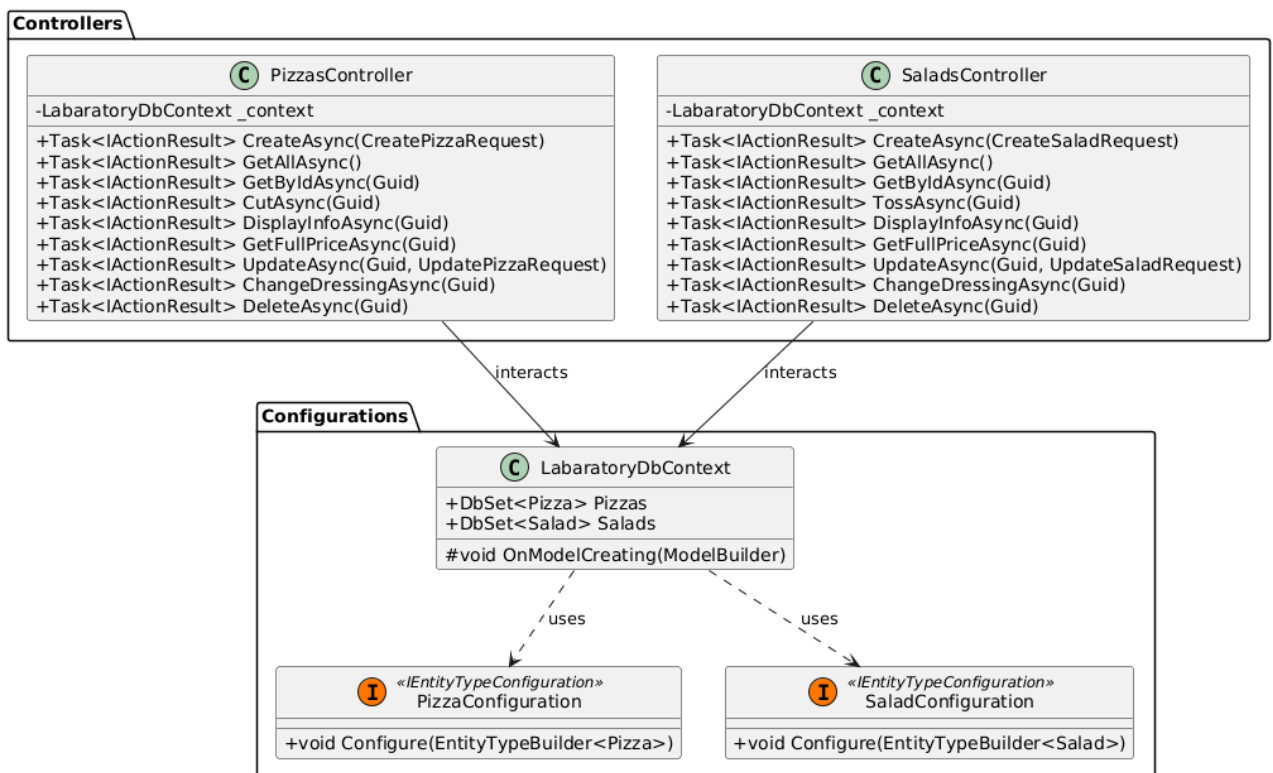


Рисунок 1 – Диаграмма классов контроллеров и конфигураций

Также разработан веб-интерфейс для взаимодействия с WEB API, экран-ные формы которого представлены на рисунках 3, 4.

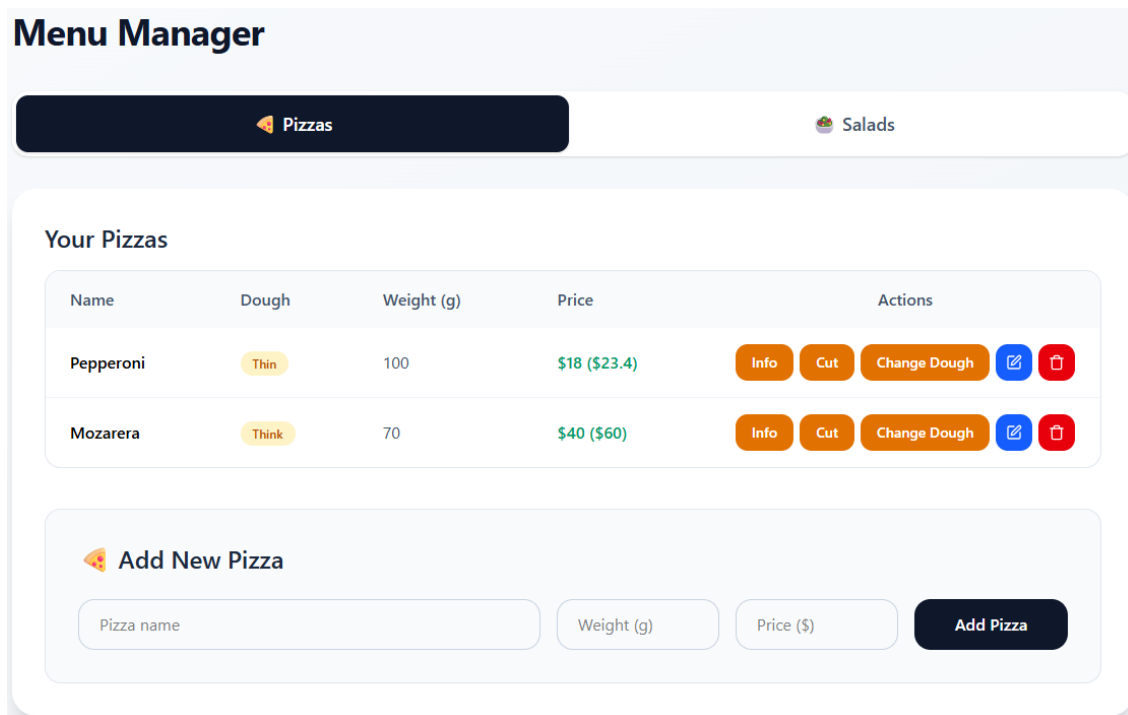


Рисунок 3 – Экранная форма отображения и создания записей

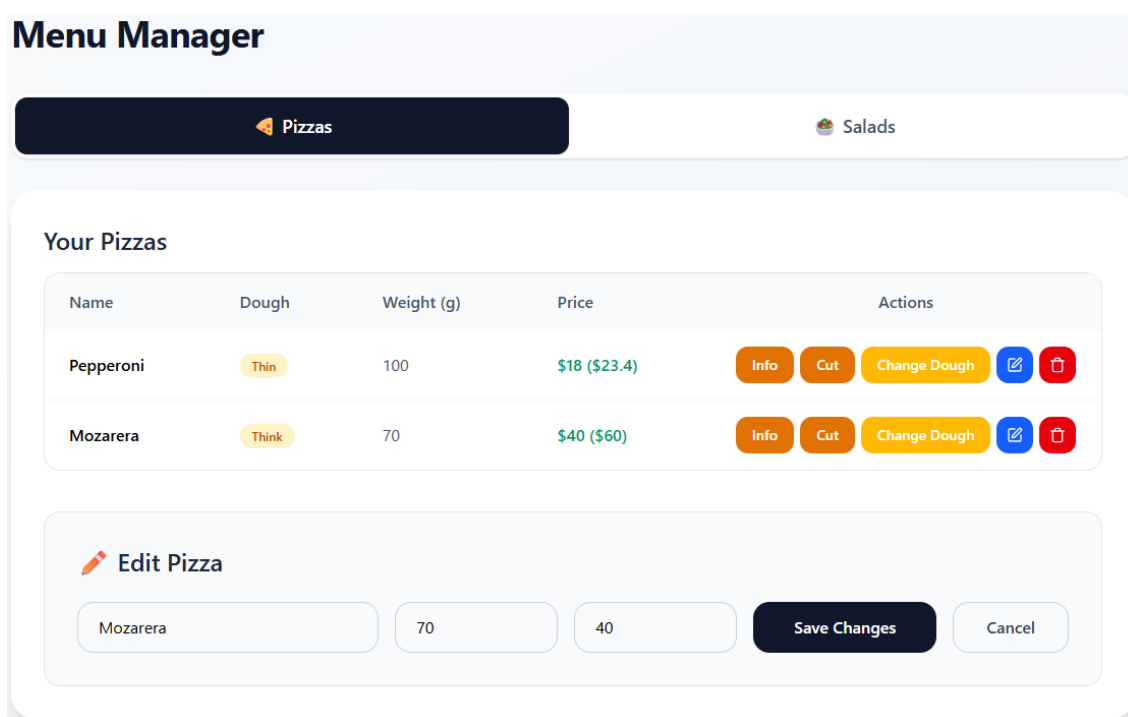


Рисунок 4 – Экранная форма редактирования записей

Вывод

В ходе выполнения лабораторной работы была разработана и реализована иерархия классов, моделирующая управление ассортиментом блюд в

заведении общественного питания. Изучены основные принципы объектно-ориентированного программирования: инкапсуляция, наследование, полиморфизм. Разработана база данных и веб-приложение, демонстрирующее применение данных принципов.